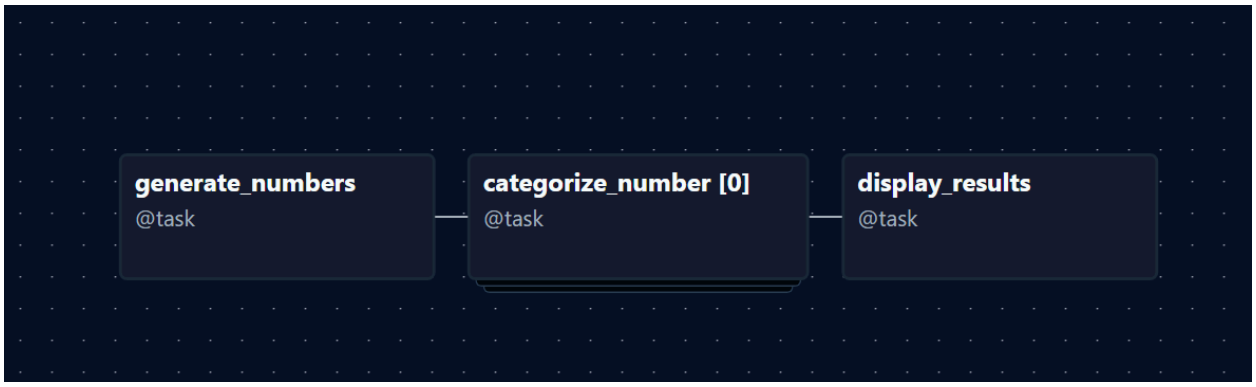
Dynamic Task Mapping Demo Pipeline

Objective

Create an Apache Airflow DAG that:

1. Generates 10 random numbers between 1 and 100.
2. Passes the generated list to the next task using XCom.
3. Uses Dynamic Task Mapping to categorize each number into segments:
    - **Great** → Number >= 80
    - **Average** → Number between 40 and 79
    - **Poor** → Number < 40
4. Displays the number along with its segment classification.

Architecture Flow



Concepts Used

Concept	Description
DAG	Defines workflow structure
XCom	Passes data between tasks
TaskFlow API	Simplifies Python-based task creation
Dynamic Task Mapping	Creates tasks dynamically at runtime
expand()	Generates mapped tasks automatically

```python
from airflow import DAG
from airflow.sdk import task
import pendulum
import random

# DAG Definition
with DAG(
    dag_id="dynamic_number_pipeline",
    start_date=pendulum.yesterday(),
    schedule=None,
    catchup=False,
    tags=["demo", "dynamic-task"],
) as dag:

# Task 1 - Generate Random Numbers

    @task
    def generate_numbers():
        """
        Generate 10 random numbers between 1 and 100
        """
        numbers = [random.randint(1, 100) for _ in range(10)]
        print(f"Generated Numbers: {numbers}")
        return numbers

# Task 2 - Categorize Number - Dynamic Task Mapping

    @task
    def categorize_number(number):
        """
        Categorize number into Great, Average, Poor
        """
        if number >= 70:
            segment = "Great"

        elif number >= 40:
            segment = "Average"

        else:
            segment = "Poor"

        result = {
            "number": number,
            "segment": segment
        }

        print(result)
        return result

 # Task 3 - Display Final Results

    @task
    def display_results(results):
        """
        Display all categorized results
        """
        print("\nFinal Results:\n")

        for item in results:
            print(
                f"Number: {item['number']} --> Segment: {item['segment']}"
            )
    # Workflow Generate list of numbers
    numbers = generate_numbers()
    # Dynamic task mapping
    categorized_results = categorize_number.expand(
        number=numbers
    )
    # Final display task
    display_results(categorized_results)
```

Step-by-Step Explanation

---

1. Import Required Libraries

```
from airflow import DAG
from airflow.decorators import task
from datetime import datetime
import random
```

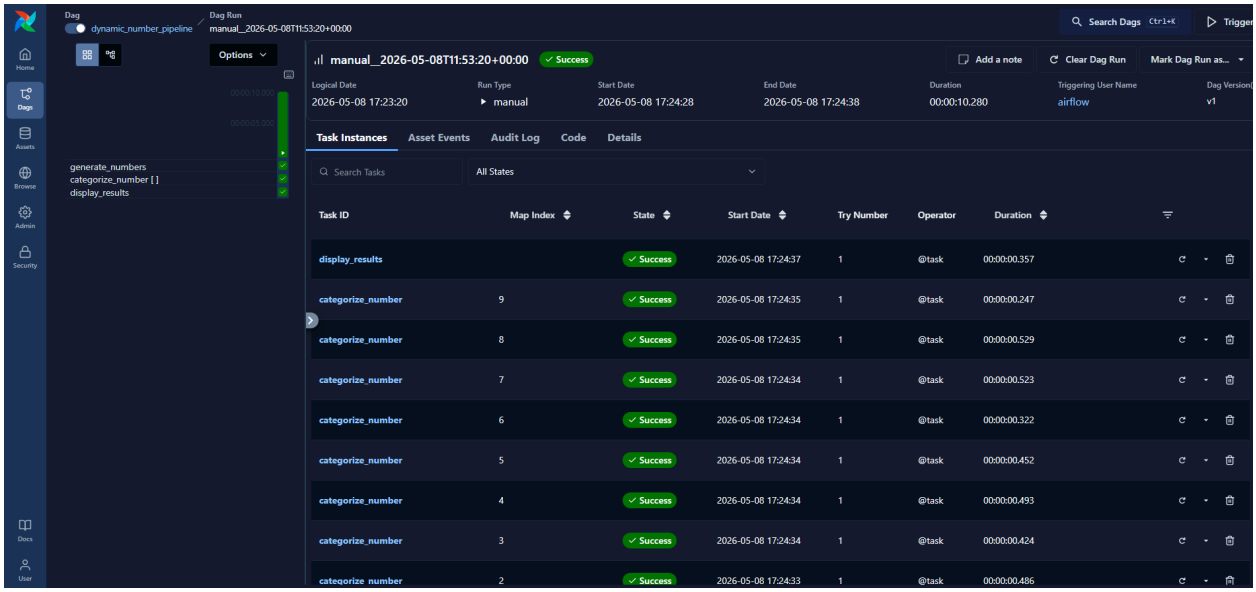
Purpose

Library	Usage
DAG	Creates workflow
task	Creates TaskFlow API tasks
datetime	Defines DAG start date
random	Generates random numbers

2. Define DAG

Parameters

Parameter	Meaning
dag_id	Unique DAG name
start_date	DAG start date
schedule_interval=None	Manual trigger
catchup=False	Prevents backfill
tags	UI grouping



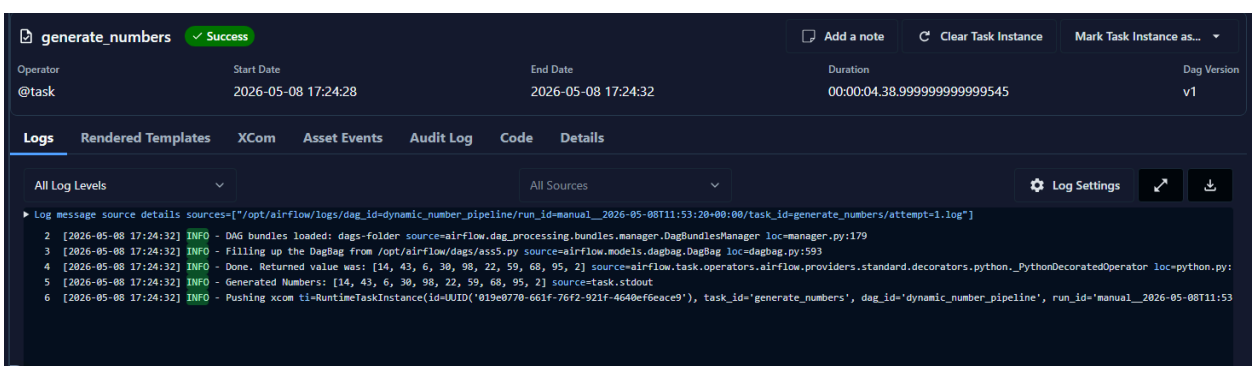
3. Generate Random Numbers Task

```
@task
def generate_random_numbers():
    numbers = [random.randint(1, 100) for _ in range(10)]
    return numbers
```

What Happens

- Generates 10 random integers
- Values range from 1–100
- Returns list through XCom automatically

Example Output



4. Dynamic Task Mapping Task

Segment Rules

Range	Segment
80 - 100	Great
40 - 79	Average
1 - 39	Poor

5. Dynamic Task Creation using expand()

```
categorized_results = categorize_number.expand(
  number=random_numbers
)
```

How expand() Works

If generated numbers are:

[10, 50, 90]

Airflow dynamically creates:

categorize_number[0] → 10
categorize_number[1] → 50
categorize_number[2] → 90

Actual Runtime Tasks

Dynamic Task	Input
categorize_number[0]	10
categorize_number[1]	50
categorize_number[2]	90

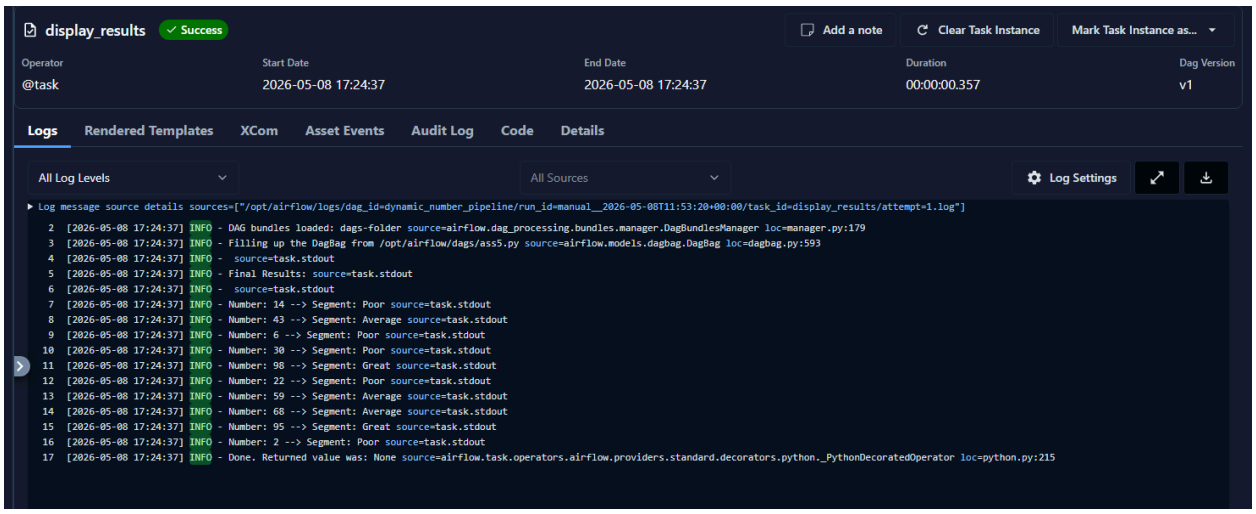
This is called **Dynamic Task Mapping**.

6. Display Final Results

Purpose

- Collects all mapped task outputs
- Prints final categorized results

Example Output



XCom Working in This DAG

What is XCom?

XCom (Cross Communication) allows tasks to share data.

Data Flow

Step 1

generate_random_numbers()

Returns:

[15, 82, 45]

Stored automatically in XCom.

Step 2

categorize_number.expand(number=random_numbers)

Airflow reads XCom list and creates dynamic tasks.

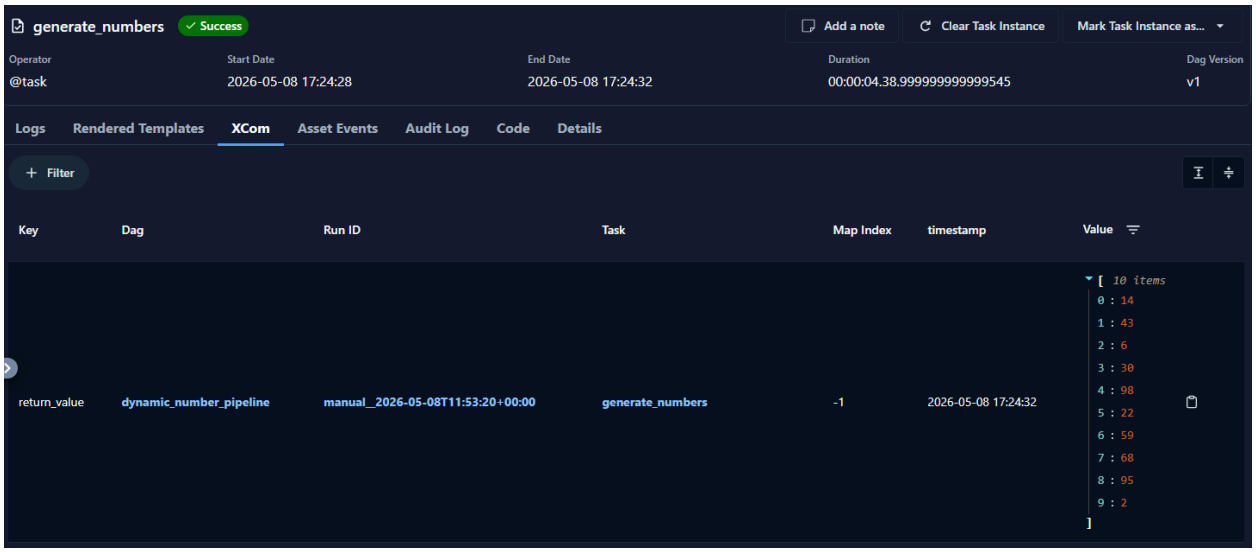
Step 3

Each mapped task returns:

```
{
 "number": 82,
 "segment": "Great"
}
```

Step 4

`display_results()` receives all mapped outputs as a list.



Dynamic Task Mapping Internals

Traditional Approach

Without dynamic mapping:

```
task1 = categorize_number(10)
task2 = categorize_number(20)
```

task3 = categorize_number(30)

Problem:

- Manual coding
- Not scalable

Dynamic Mapping Approach

categorize_number.expand(number=numbers)

Benefits:

- Automatic task creation
- Scalable
- Cleaner code
- Runtime flexibility

Expected Airflow UI Flow

```
generate_random_numbers
      ↓
categorize_number[0]
categorize_number[1]
categorize_number[2]
...
categorize_number[9]
      ↓
display_results
```

Sample Execution

Generated Numbers

[91, 12, 45, 67, 32]

Dynamic Task Outputs

Number	Segment
91	Great
12	Poor
45	Average
67	Average
32	Poor

Advantages of Dynamic Task Mapping

Benefit	Explanation
Scalability	Handles large datasets dynamically
Less Code	No manual task creation
Runtime Flexibility	Tasks created during execution
Parallelism	Tasks run simultaneously
Better Monitoring	Individual task visibility

Real-Time Industry Use Cases

Use Case	Example
File Processing	One task per file
API Calls	One task per API request
Data Validation	One task per dataset
ETL Pipelines	Parallel table processing

Conclusion

This DAG demonstrates:

- TaskFlow API
- XCom communication
- Dynamic Task Mapping
- Parallel execution
- Runtime task generation